Лабораторная работа №,1 по курсу дискретного анализа: сортировка за линейное время

Выполнил студент группы 08-215 МАИ *Матвеев Семен.*

Условие

Требуется разработать программу, осуществляющую ввод пар «ключ-значение», их упорядочивание по возрастанию ключа указанным алгоритмом сортировки за линейное время и вывод отсортированной последовательности.

Вариант задания:

- Алгоритм: Карманная сортировка
- Тип ключа: число от 0 до $2^{64} - 1$
- Тип значения: строки переменной длины (до 2048 символов)

Метод решения

Для сортировки набора пар (ключ, значение) применяется карманная сортировка.

Основная идея:

1. Исходный массив разбивается на несколько корзин (buckets) на основе значения ключа.
2. Количество корзин выбирается равным $\lfloor \sqrt{n} \rfloor$, где n — число элементов. Такой выбор даёт компромисс между количеством корзин и размером каждой из них.
3. Для каждого элемента вычисляется номер корзины по формуле:

$$bucketIndex = \left\lfloor \frac{key - \min}{1 + \frac{\max - \min}{buckets}} \right\rfloor$$

   Формула обеспечивает равномерное распределение элементов по корзинам при равномерном распределении ключей (оно является требованием к данным для эффективной работы bucket sort).
4. Каждая корзина сортируется независимо. Для этого используется сортировка слиянием (merge sort), гарантирующая устойчивость и сложность $O(k \log k)$ в худшем случае для корзины размера k.
5. После сортировки всех корзин элементы собираются обратно в единый массив в порядке возрастания ключей.

При равномерном распределении ключей размер каждой корзины составляет примерно $\sqrt{n}$, поэтому общая трудоёмкость близка к $O(n)$, так как сортировка каждой корзины даёт $O(\sqrt{n} \log \sqrt{n})$, а сумма по всем корзинам — $O(n \log \sqrt{n})$.

Описание программы

Шаблон класса TVector<T> — динамический массив с ручным управлением памятью. Поддерживает копирование, перемещение, доступ по индексу, изменение размера, добавление элемента в конец. Особенность: в методе push_back используется placement new, что заметно уменьшило время выполнения программы в процессе отладки.

Класс TItem — хранит ключ (size_t key) и значение (TVector<char> value), имеет конструкторы, операторы сравнения, геттеры полей.

Перегруженные операторы ввода и вывода — для TItem и TVector<T> перегружен оператор вывода. Перегружен оператор ввода для TVector<T>, также есть специализированный оператор ввода для TVector<char>.

Вспомогательные функции — MakeTVector (для создания TVector<char> из C-строки), GenerateRandomInput (для генерации случайных входных данных), шаблонная RunBenchmark (для теста производительности на случайных выходных данных), BubbleSort (обменная сортировка для сравнения с BuckerSort в рамках RunBenchmark) и шаблонные Min и Max.

Функции сортировки — Merge (слияние двух отсортированных половин), MergeSort (итеративная сортировка слиянием всего массива), BucketSort (основная функция карманной сортировки).

Функция main — читает входные данные до конца потока, сортирует их вызовом BucketSort, затем выводит каждую пару на экран. Опционально, раскомментировав блоки кода, можно использовать случайный ввод вместо ручного пользовательского и провести бенчмарк функции сортировки.

Дневник отладки

1. Реализовал каркас программы - ввод данных в виде набора пар (ключ, значение), сохранение их в std::vector, их сортировка с помощью std::sort, вывод на экран. Пришлось осознать, что в выводе надо использовать символы табуляции.

2. Реализовал функцию сортировки MergeSort и вспомогательные функции Merge, Min и Max. Возникали ошибки выхода за границы массива из-за неверных индексов в циклах. Исправлено изменением логики использования переменных left, mid, right.

3. Реализовал функцию BucketSort. Возникали ошибки выхода за границы массива из-за некорректного вычисления переменной bucketIndex. Я ориентировался на нерекурсивную реализацию карманной сортировки в конспектах ИТМО, но там эта формула была чрезвычайно странной. Сначала боролся с переполнением, потом с логикой вычисления (классическая нормализация и добавление 1, чтобы гарантировать попадание максимального ключа в последнюю корзину).

4. Реализовал вспомогательные функции GenerateRandomInput (которая также помогала при отладке), RunBenchmark и BubbleSort для проведения тестов производительности функции BucketSort.

5. Самое сладкое. Было необходимо избавиться от использования STL (в моем случае std::pair, std::vector, std::string). Для этого были реализованы классы TItem, TVector и все их методы. Также понадобилось перегрузить operator» и operator« для ввода и вывода объектов класса TVector соответственно. Ошибок было несчетное множество, все уже и не упомнишь. Могу лишь сказать, что я в какой-то момент напрягся настолько, что единолично увеличил уровень общемировой напряженности, чем спровоцировал очередной конфликт на Ближнем Востоке. За что искренне прощу прощения.

Тест производительности

Для оценки эффективности реализованных алгоритмов был проведён эксперимент на случайных данных. Генерация входных наборов выполнялась функцией GenerateRandomInput с фиксированной строкой-значением 'stroka' и равномерно распределёнными 64-битными ключами (диапазон $[0, 2^{64} - 1]$).

Время сортировки измерялось с помощью std::chrono::high_resolution_clock; для каждого объёма данных проводилось три запуска, результат усреднялся.

Исследовались следующие алгоритмы:

- BucketSort — карманная сортировка с количеством корзин $\lfloor \sqrt{n} \rfloor$ и внутренней сортировкой слиянием;
- BubbleSort — пузырьковая сортировка (включена в программу для демонстрации контраста сложности).

Из-за квадратичной сложности пузырьковой сортировки её тестирование выполнялось только для небольших n (до 50,000), чтобы время измерений оставалось приемлемым.

Результаты измерений приведены в таблице.

Количество элементов	**BucketSort, мс**	**BubbleSort, мс**
1,000	0.69	8.23
5,000	3.90	207.26
10,000	8.33	841.87
50,000	41.64	21038.40
100,000	89.26	–
500,000	500.58	–
1,000,000	1077.96	–

Для BucketSort прослеживается близкая к линейной зависимость времени от размера входных данных: при увеличении n в 10 раз (с 100,000 до 1,000,000) время выполнения увеличивается в 12.07 раза, что согласуется с теоретической оценкой $O(n)$ при равномерном распределении ключей.

Пузырьковая сортировка демонстрирует ожидаемый квадратичный рост: при увеличении n в 5 раз (с 1,000 до 5,000) время выполнения увеличивается в 25 раз, что соответствует $O(n^2)$.

Полученные результаты подтверждают корректность реализации и соответствие заявленной сложности.

Недочёты

- При вычислении индекса корзины используется деление с плавающей точкой (long double), что может привести к небольшим погрешностям, с которыми, вроде бы, получилось совладать.
- Не обрабатывается случай пустого входного потока — программа завершится без вывода, что допустимо, но можно было бы добавить проверку.
- Отсутствует явная проверка на превышение максимальной длины строки (2048 символов), так как автор программы доверяет условию. В текущей реализации строка может быть любой длины, что потенциально может привести к исчерпанию памяти.

Выводы

В результате выполнения лабораторной работы реализована программа сортировки пар «ключ-значение» методом карманной сортировки. Ключи — 64-битные целые числа, значения — строки переменной длины. Все компоненты программы разработаны без использования STL, что позволило глубже изучить управление памятью и алгоритмы.

В процессе отладки были устранены ошибки выхода за границы массивов, некорректного вычисления индексов корзин и ввода строк. Применение placement new позволило сократить накладные расходы на копирование.

Тестирование на равномерно распределённых данных подтвердило близкую к линейной зависимость времени работы от объёма входных данных ($O(n)$), что соответствует теоретической оценке для карманной сортировки. Для сравнения пузырьковая сортировка показала ожидаемый квадратичный рост, что косвенно подтверждает корректность измерений.

Таким образом, задача решена в полном объёме, полученные навыки могут быть использованы при разработке эффективных алгоритмов сортировки в условиях ограниченных ресурсов.